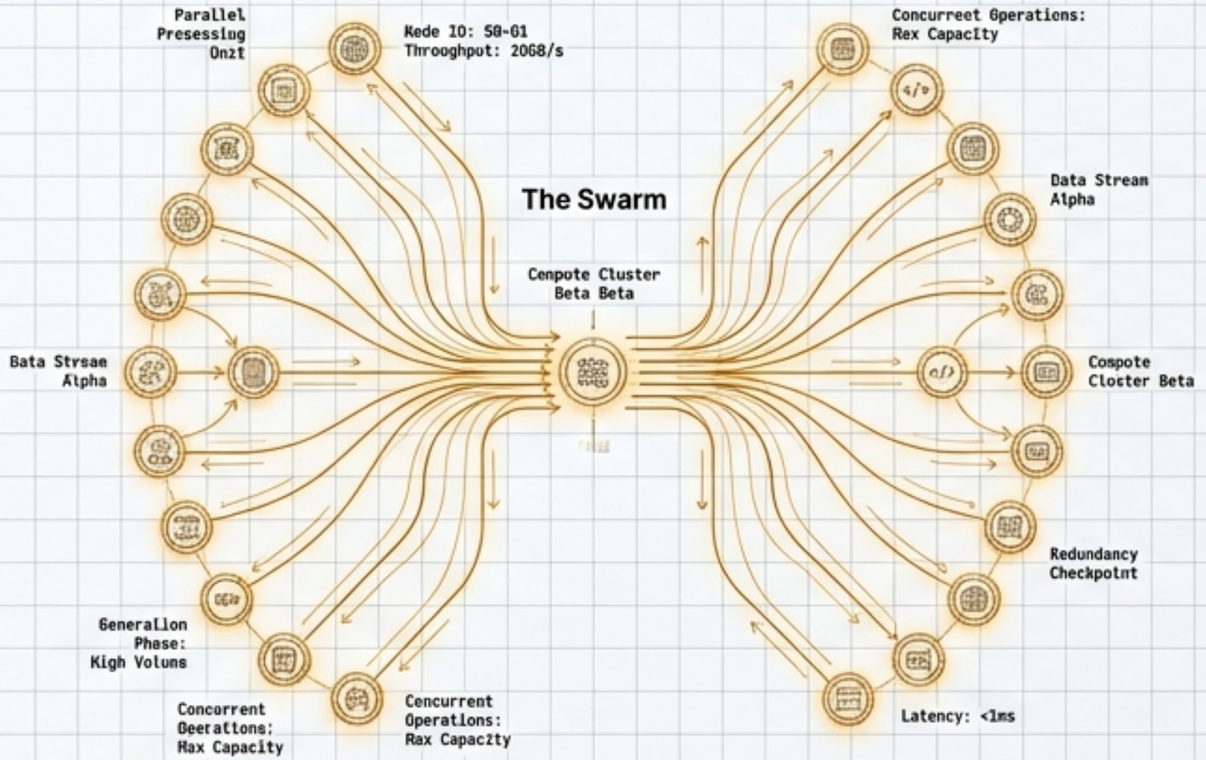
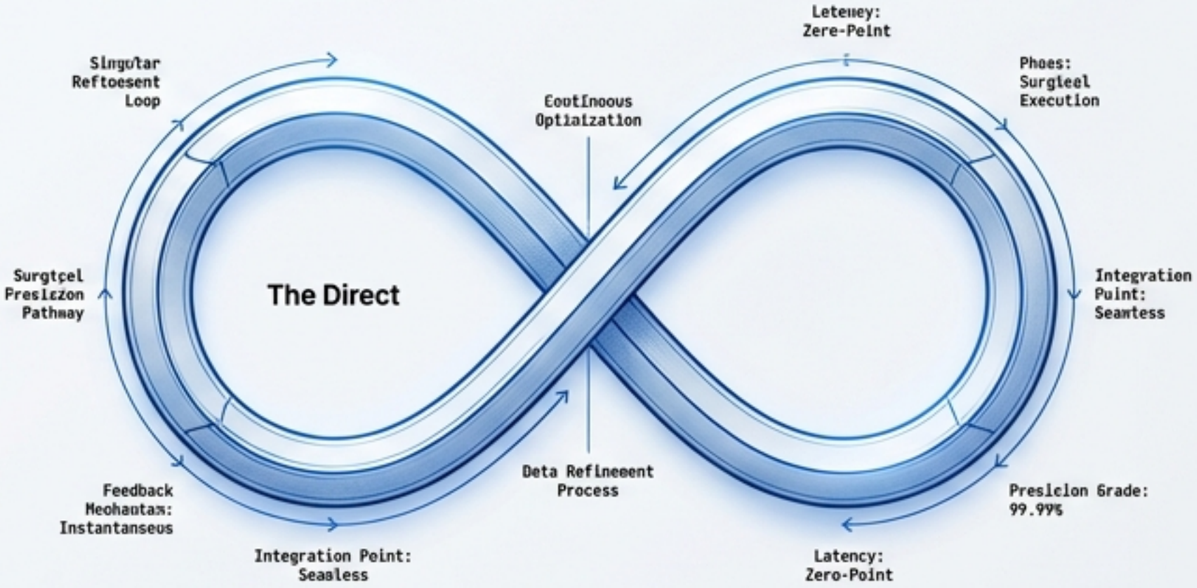
# Architecture by Phase: CRM Development

## From Parallel Generation to Surgical Precision




Date: 26 January 2026
Document Type: Technical Architecture & Workflow Diagrams
Version: 1.2.4-RC
Status: APPROVED
Engineered by: SYSTEM ARCHITECTS GMBH

# The Four-Phase Execution Strategy

**AMBER SWARM FLOW**
PHASE 01+02

**COBALT DIRECT MODE**
PHASE 03+04

1 | 2 | 3 | 4

SWARM_ID: SEE_81
STATUS: ACTIVE

AGENTS_PARALLEL: 5+3

DIRECT_ID: BOT_83
TARGET: MAINTENANCE

TOOL: CLAUDE_EDIT
STATUS: DEPLOY

## Phase 1: Create CRM API

**Objective:** Greenfields Creation
**Mode:** Claude-Flow Swarm

## Phase 2: Create UI + Docs

**Tooling:** 5 & 3 Parallel Agents

## Phase 3: Bug Fixing

**Objective:** Brownfields Maintenance
**Mode:** Claude Direct

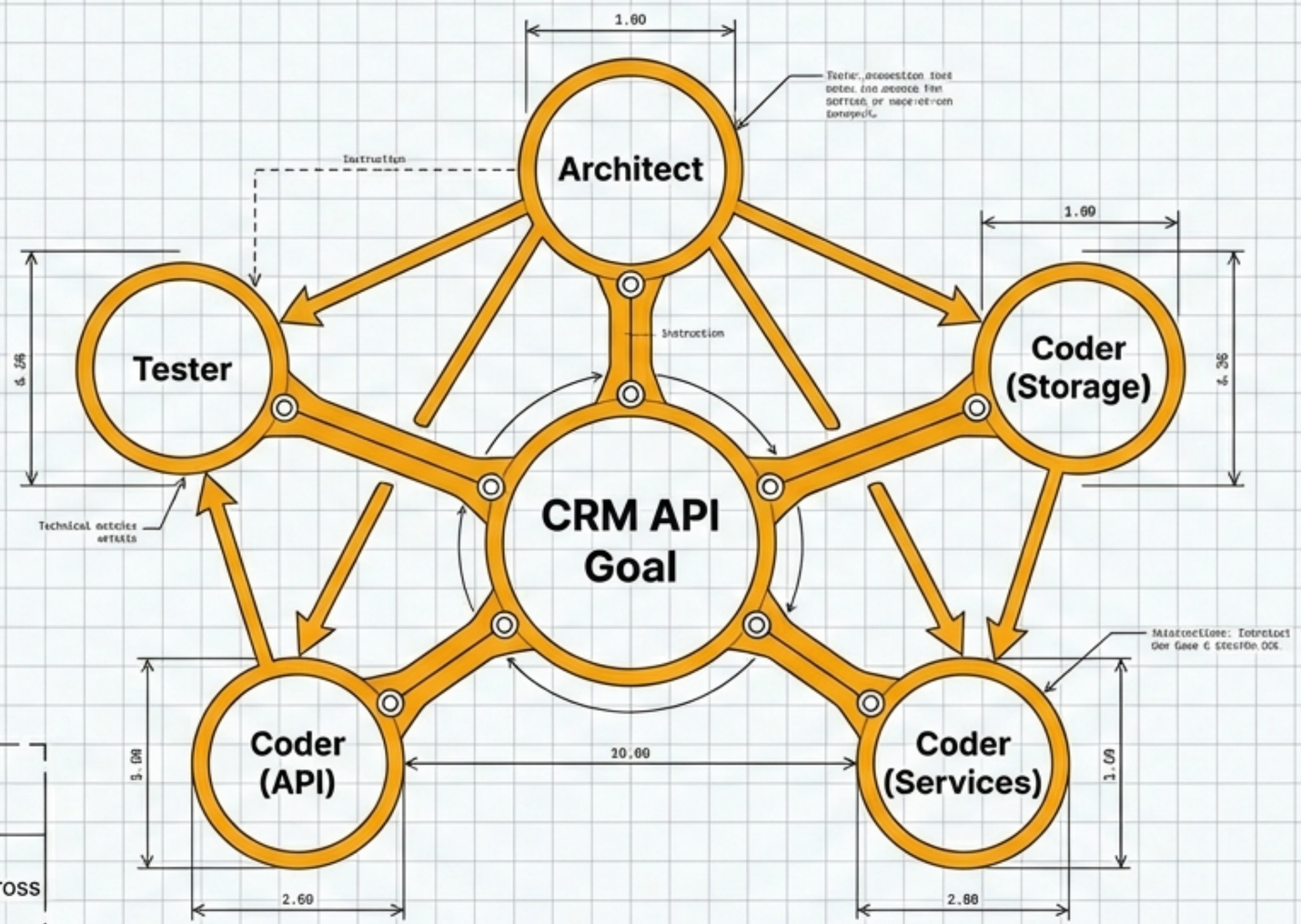
## Phase 4: New Features

**Tooling:** Claude Code with Edit Tool

SEGMENT_WIDTH: 25%
TIMELINE_SPAN: 100%
SEGMENT_WIDTH: 25%
TIMELINE_WIDTH: 100%

# Phase 1: High-Velocity API Creation

## The Swarm Era (Parallel Execution)




**Execution Mode:**
SWARM (5 Parallel Agents)

**Total Output:**
~1,800+ Lines of Code generated across storage, services, and API layers.

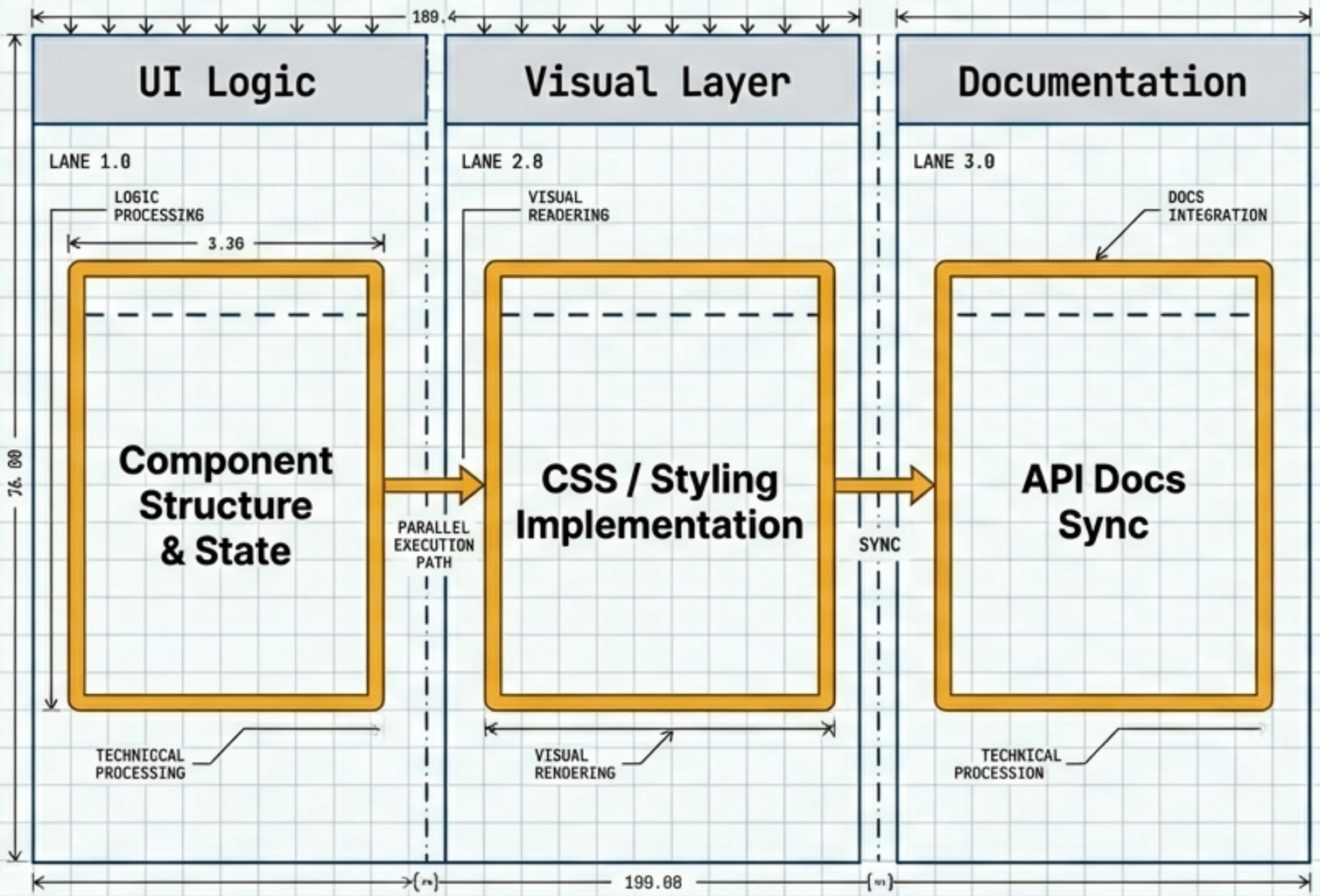
# Phase 1 Deep Dive: Agent Orchestration

| AGENT ROLE | MODEL | CORE TASK | OUTPUT & VOLUME |
|---|---|---|---|
| Agent 1 (System Architect) | Sonnet | Design type system | src/types/index.ts (129 lines) |
| Agent 2 (Coder) | Sonnet | Build storage layer | src/storage/Store.ts, index.ts (116 lines) |
| Agent 3 (Coder) | Sonnet | Implement services | src/services/*.ts (~400 lines) |
| Agent 4 (Coder) | Sonnet | Create REST API | src/api/*.ts (~700 lines) |
| Agent 5 (Tester) | Sonnet | Write unit tests | tests/*.ts (~500 lines) |

**Dependency Flow:** Architect defines types first; **Coders build simultaneously** based on strict type definitions.

# Phase 2: Interface & Documentation

## Applying the Swarm to the Frontend




**Execution Mode:**

SWARM (3 Parallel Agents)

**Model Selection Strategy:**

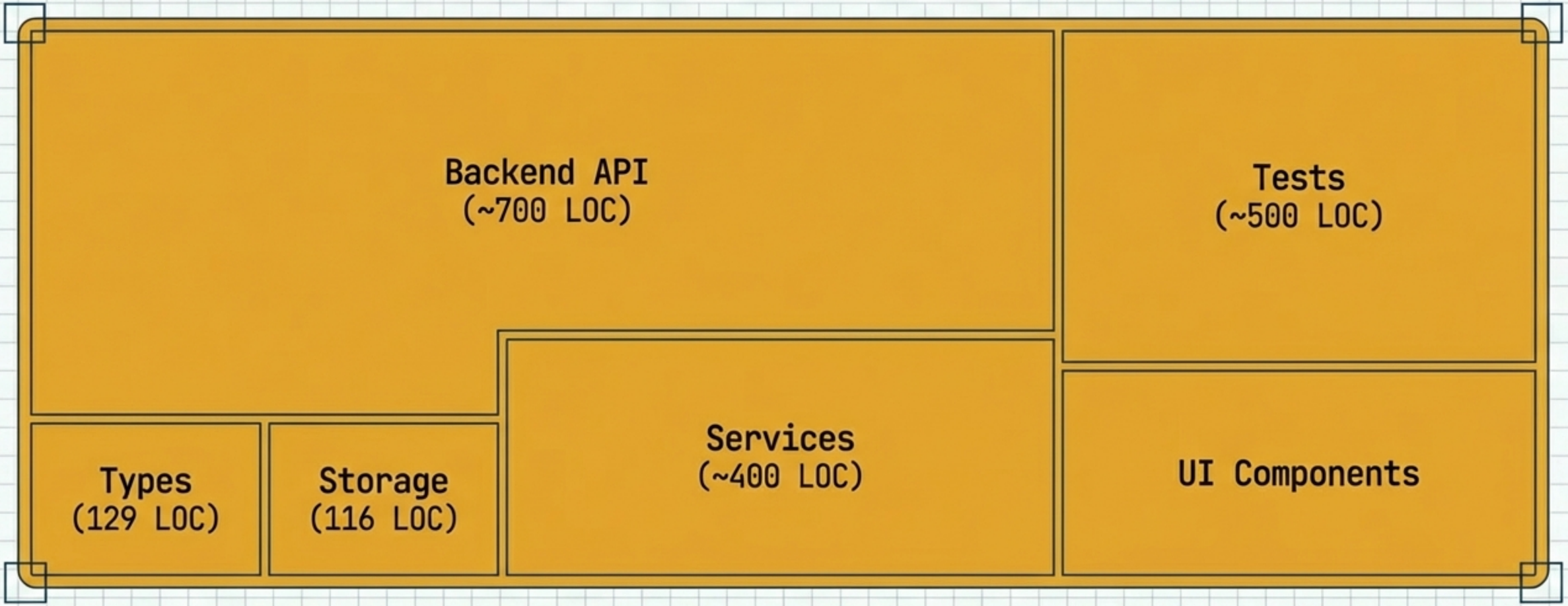
Visual tasks require spatial reasoning models. Logical tasks utilise reasoning-heavy models. Parallel execution reduces context switching costs.

**Deliverables:**

```
├── src/                    [3.2 KB]
│   ├── components/         [3.2 KB]
│   │   ├── ui/             [1.1 KB]
│   │   └── layout/         [1.1 KB]
└── docs/                   [1.1 KB]
```

# The Swarm Retrospective

## Phases 1 & 2 Analysis




**Capability:** High volume, parallel execution.
**Result:** A functional, testable MVP consisting of a full backend API, storage layer, and frontend interface.
**Transition:** The system now exists. The challenge shifts from creation to refinement.

# The Strategic Pivot

Why Switch Modes?

## SWARM ERA


### Many Hands

- Greenfields
- Generation
- Speed

## DIRECT ERA

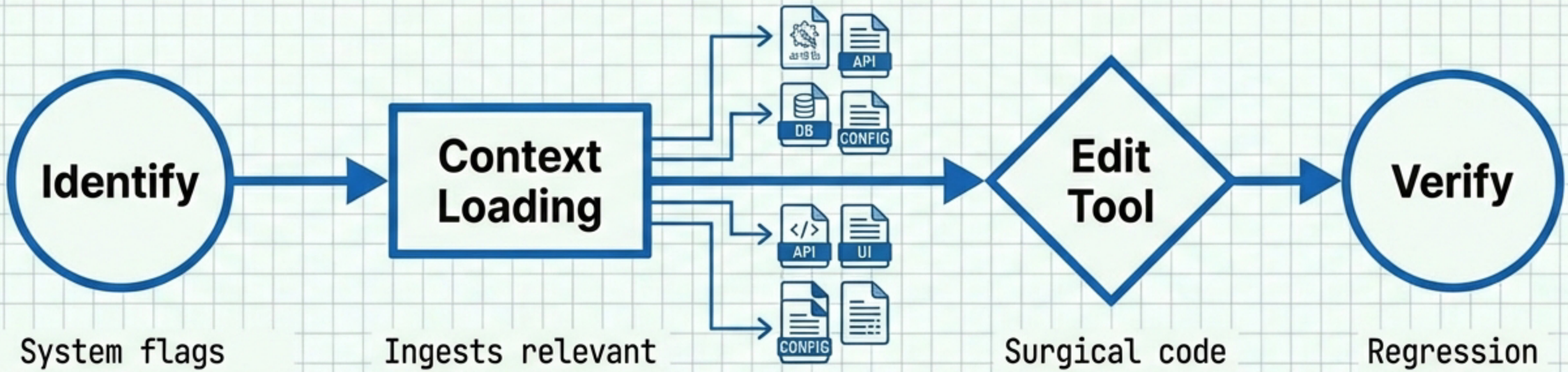

### One Mind

- Brownfields
- Maintenance
- Context

Swarms build foundations. But maintenance requires context. To fix bugs and add features without breaking dependencies, we transition to a singular, context-aware entity: Claude Direct.

# Phase 3: Stabilisation & Bug Fixing

Execution Mode: DIRECT
(Claude Code with Edit Tool)




System flags integration error.

Ingests relevant files across full stack.

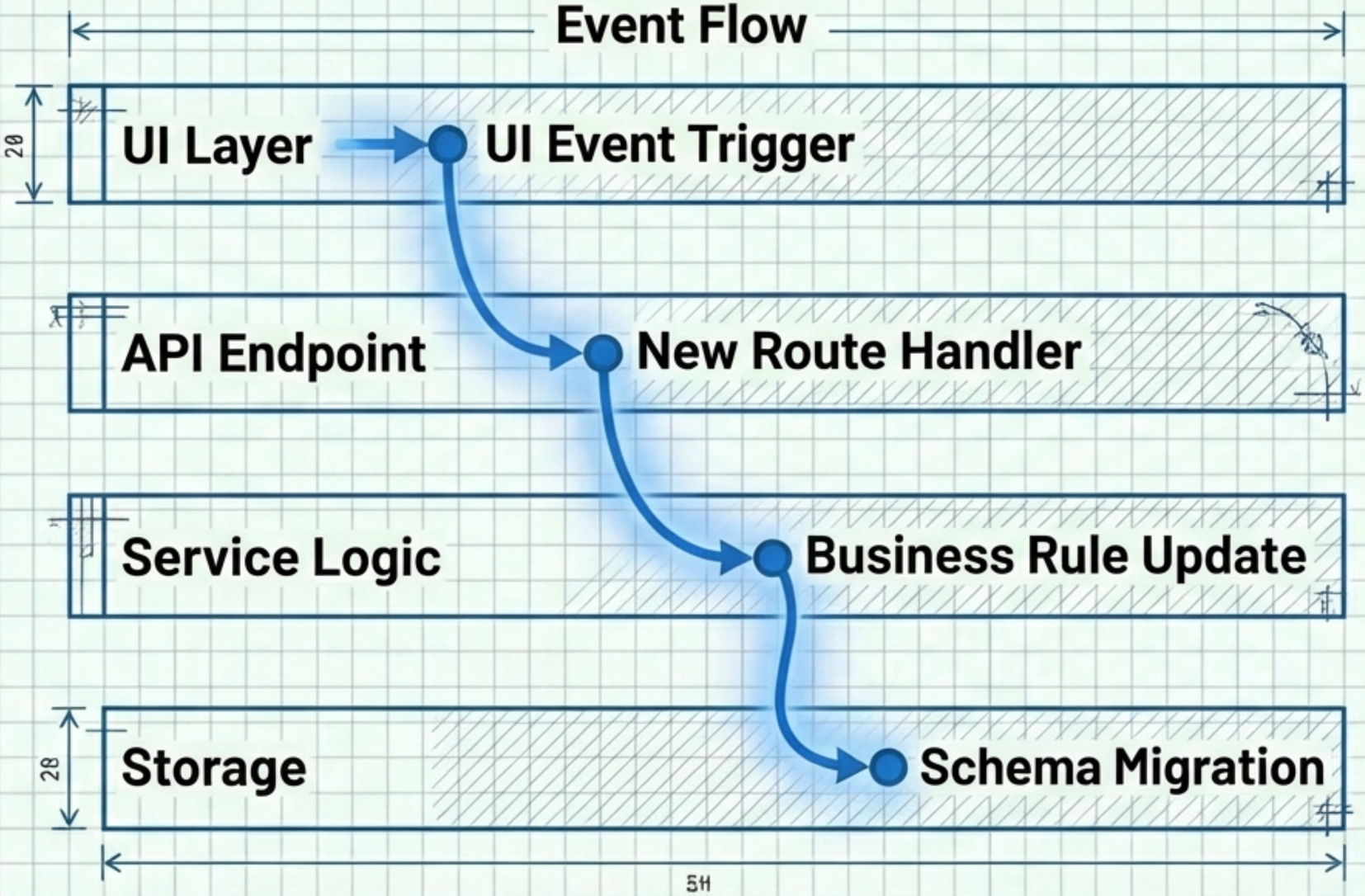
Surgical code changes.

Regression testing.

Focused on the inevitable integration friction that arises after massive parallel generation.

# Phase 4: Feature Expansion

Execution Mode: DIRECT
(Claude Code with Edit Tool)




**Key Insight:**

The Direct agent holds the 'mental map' of the entire project, allowing for complex insertions that a swarm of isolated agents might mishandle.

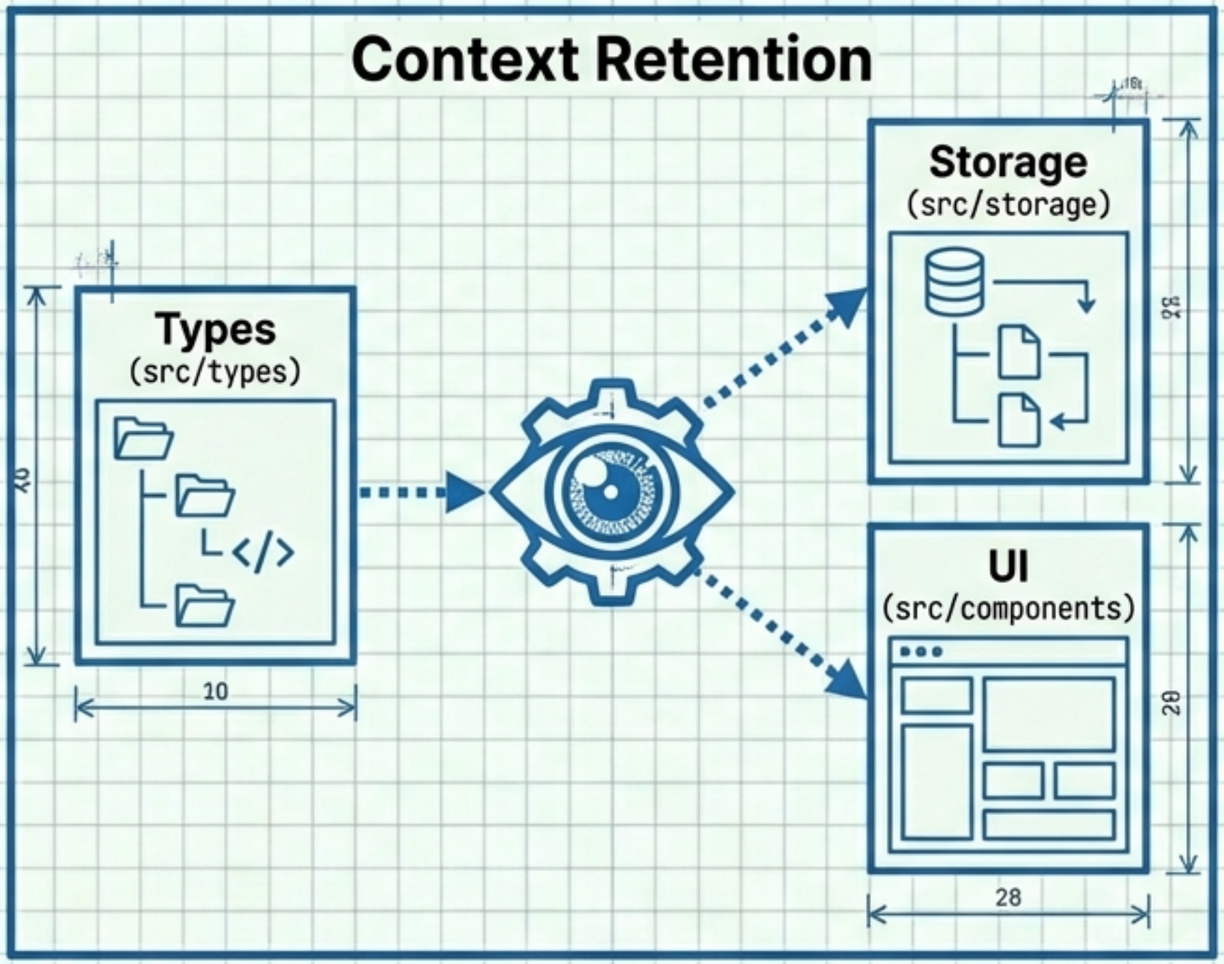
The feature is woven into the existing src/ directory without disrupting the Phase 1 & 2 foundation.

# Direct Mode Logic

## The Necessity of Context

### Why Direct Mode (Not Swarm)?

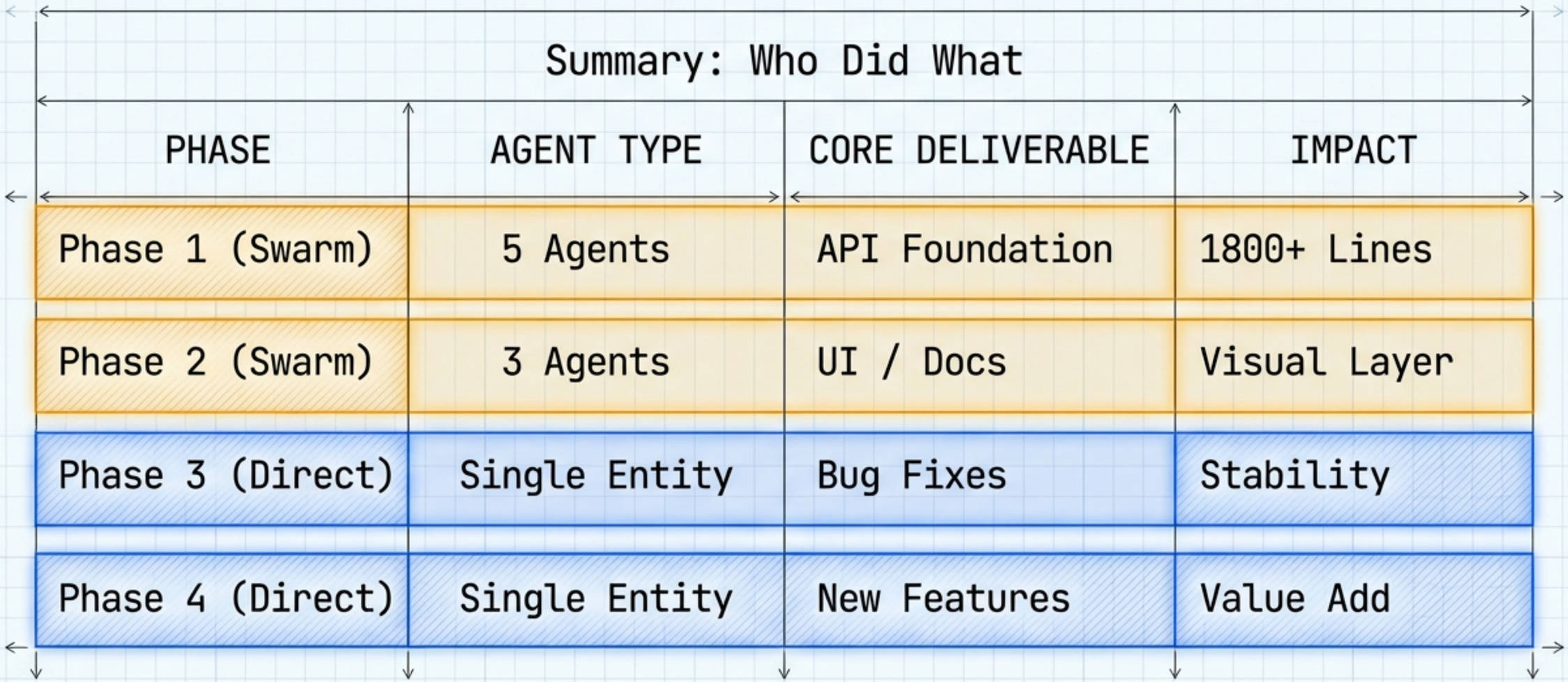
Swarms work best when tasks can be isolated. Feature expansion in an existing codebase is highly - highly coupled.




### Surgical Precision:

Direct mode avoids "hallucinating" overwrite conflicts or duplicating logic, ensuring the integrity of the architecture established in Phase 1.

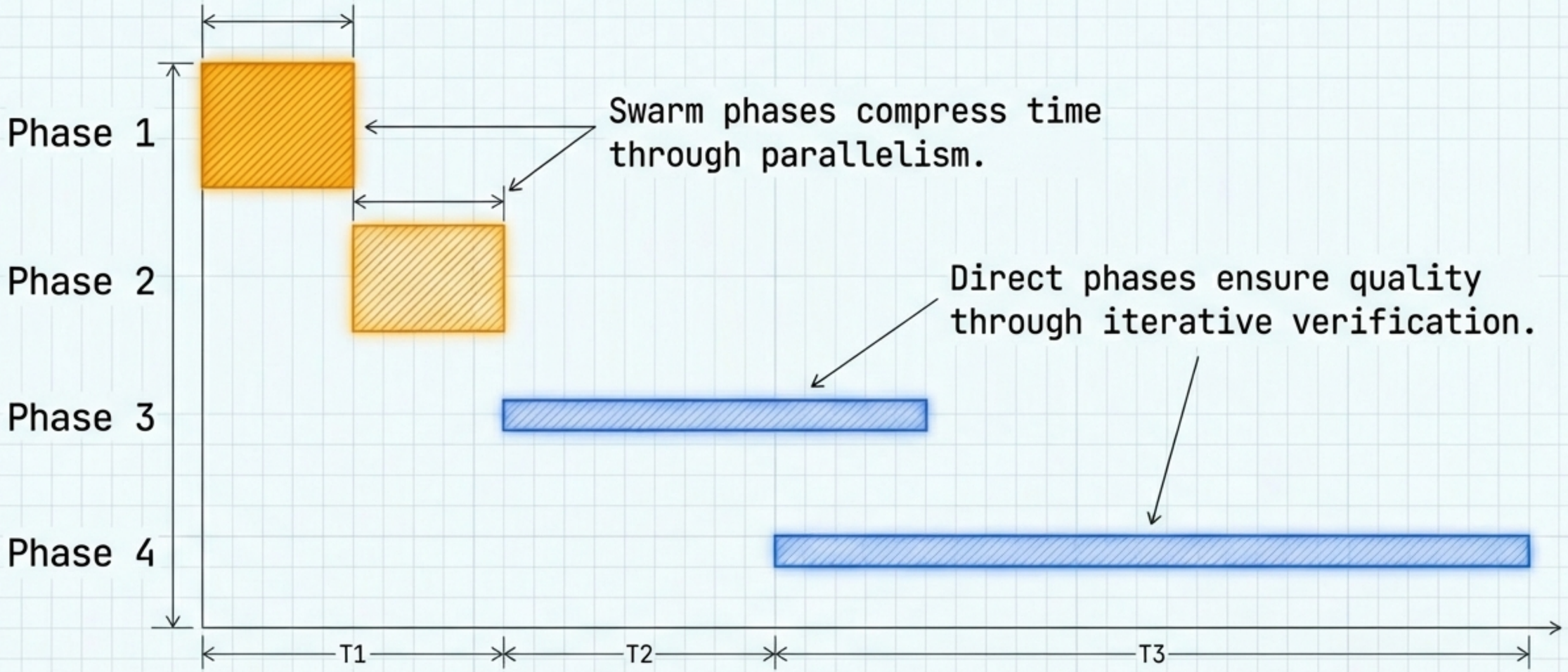
# Role & Output Breakdown



APPROVED DESIGN
2024-10-27

VERSION: 1.0 REV A

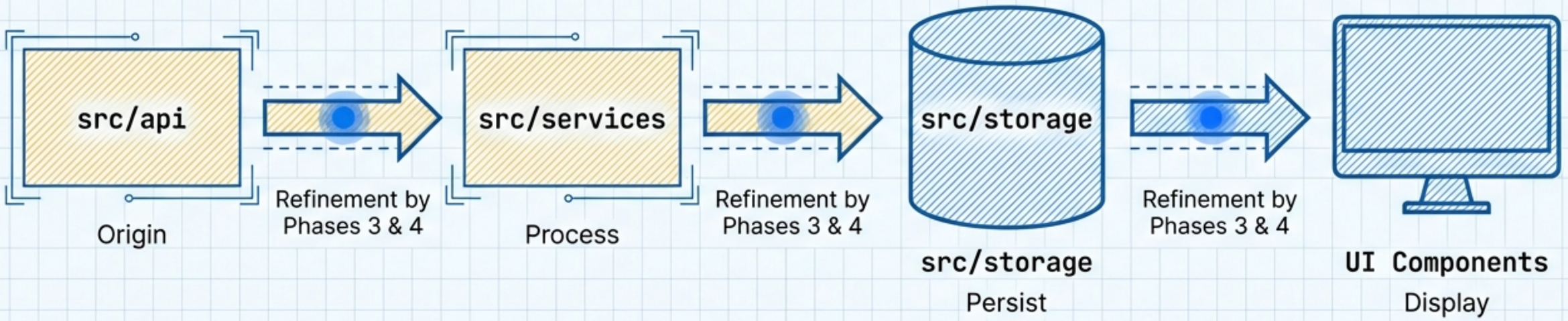
# Project Timeline




DATE: 2024-10-27

VERSION: 1.0 REV A

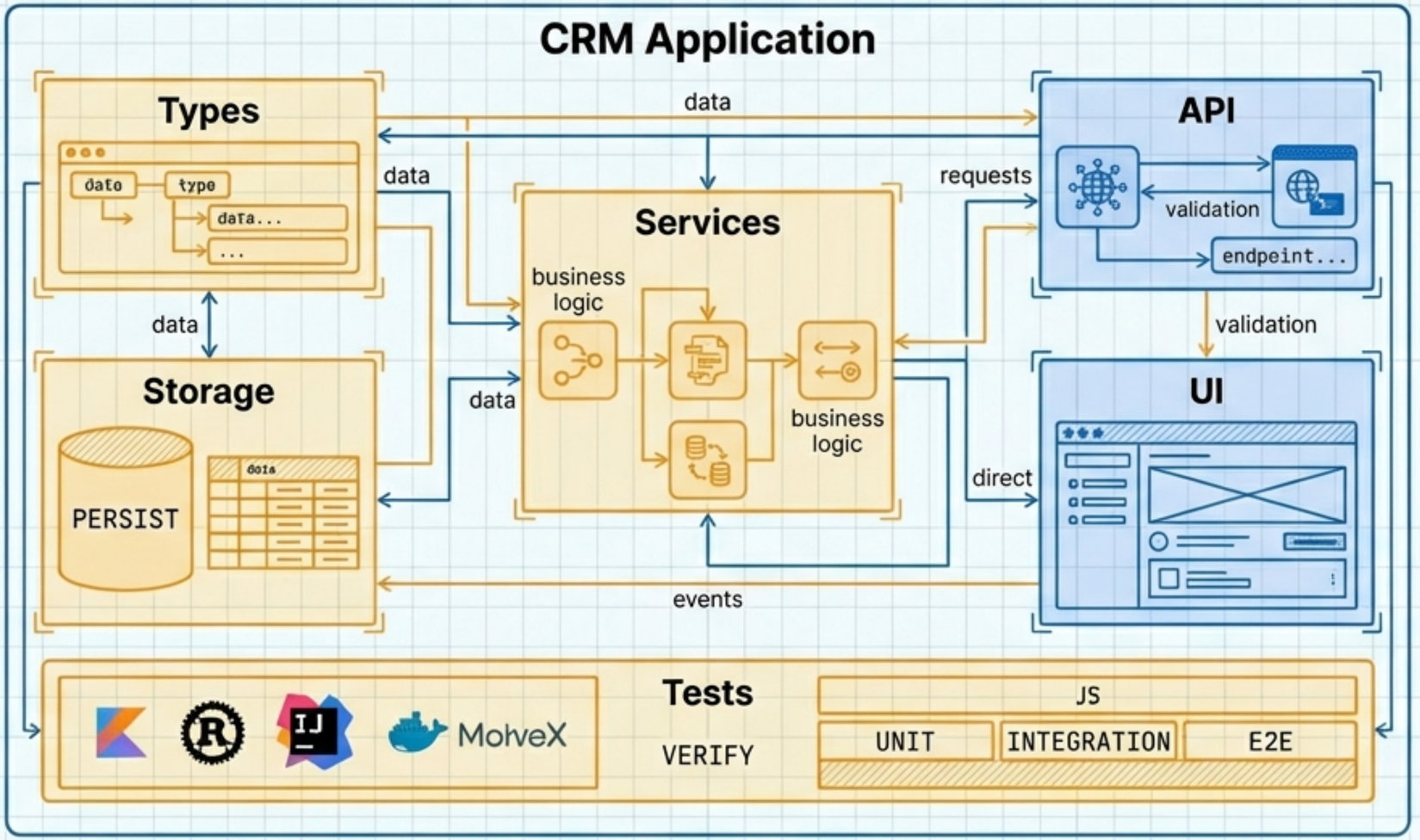
# Architectural Data Flow



APPROVED DESIGN
2024-16-27

VERSION: 1.0 REV A

# Final State Architecture




DATE: 2024-10-27

A production-ready CRM foundation, built from zero, stabilised, and expanded.

VERSION: 1.0 REV A

APPROVED DESIGN
2024-16-27

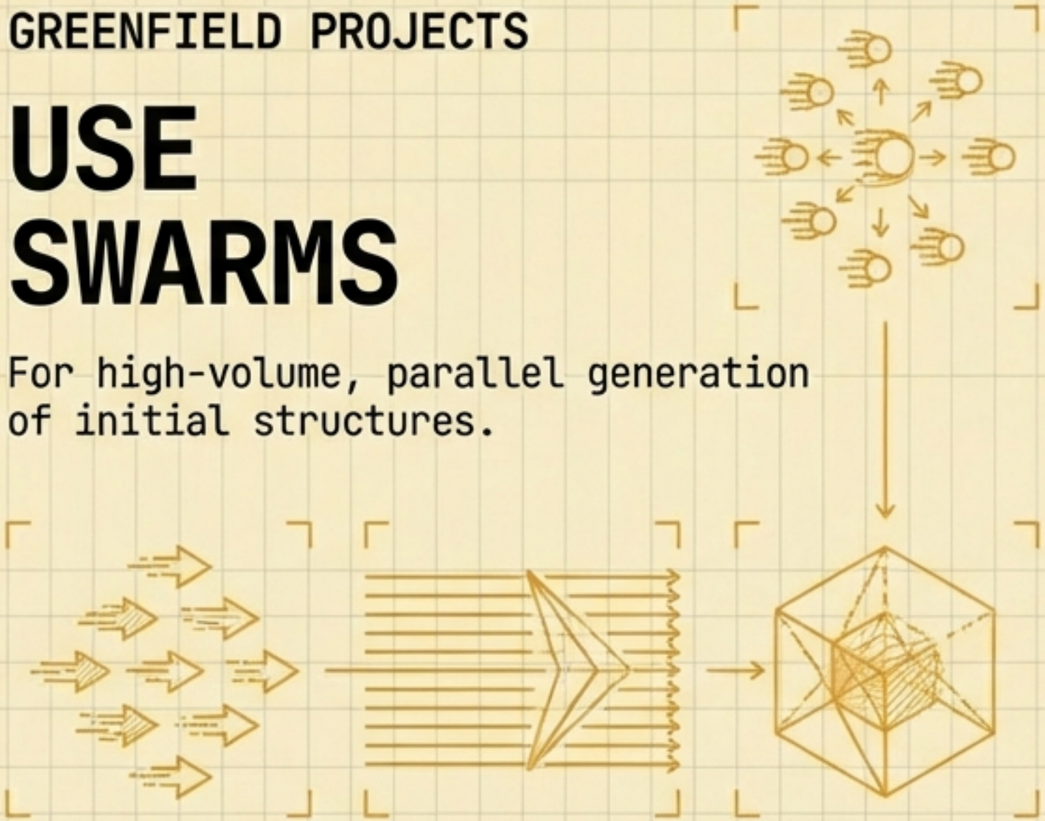
VERSZON: 1.0 REV A

# The Development Heuristic

GREENFIELD PROJECTS

## USE SWARMS

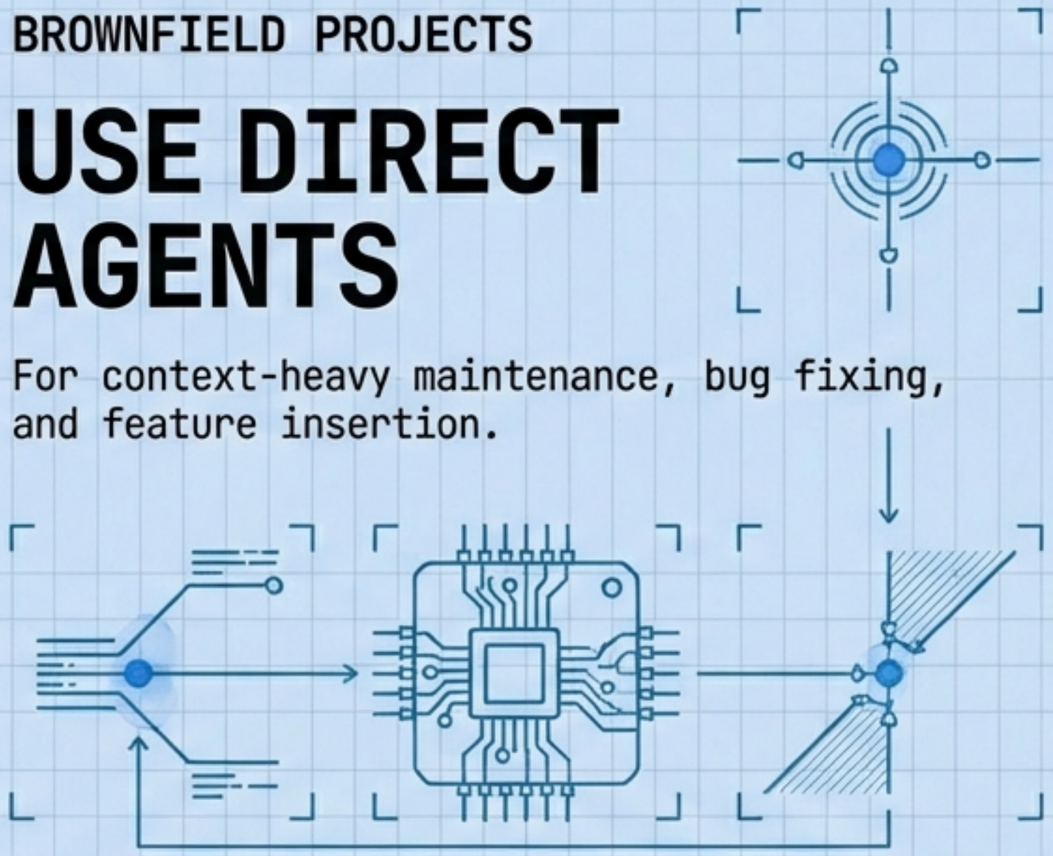
For high-volume, parallel generation of initial structures.



BROWNFIELD PROJECTS

## USE DIRECT AGENTS

For context-heavy maintenance, bug fixing, and feature insertion.



DATE: 2024-10-27

**Optimise for velocity in creation. Optimise for context in maintenance.**

VERSION: 1.0 REV A